

CM04 - Primitive Data Types

Instructions, representations, objects, operators, conversions, lifetime

Louis Ledoux

2026-07-05

Table of contents

1	Instructions and Comments	2
1.1	Main Path	2
1.2	Worked Example	3
1.3	Anecdote Or Motivation	3
1.4	Check Question	4
1.5	Backup Index	4
1.6	Backup: CM 4-5 Basic components of C language: Primitive data types, Libraries and Control structu ..	5
1.7	Backup: Instructions & Comments	5
2	Value Representation and Data Objects	6
2.1	Main Path	6
2.2	Worked Example	7
2.3	Anecdote Or Motivation	7
2.4	Check Question	8
2.5	Backup Index	8
2.6	Backup: Value representation	11
2.7	Backup: Data object definition	12
2.8	Backup: Primitive data types: Type specifiers	12
2.9	Backup: Primitive data types: Size	13
2.10	Backup: Primitive data types: Min values	14
2.11	Backup: Primitive data types: Float	15
2.12	Backup: Defining primitive data objects: Examples	15
3	Operators	16
3.1	Main Path	16
3.2	Worked Example	17
3.3	Anecdote Or Motivation	17
3.4	Check Question	18
3.5	Backup Index	18
3.6	Backup: Operators	21
3.7	Backup: Arithmetic operators	22
3.8	Backup: Relational operators	23
3.9	Backup: Logical operators	24
3.10	Backup: Bitwise operators	25
3.11	Backup: Assignment operators	26
3.12	Backup: Common error: Assignment Vs Testing	27
4	and == are different !	27
4.1	Backup: Let's...	28
5	Conversions	28
5.1	Main Path	29
5.2	Worked Example	29

5.3	Anecdote Or Motivation	30
5.4	Check Question	30
5.5	Backup Index	31
5.6	Backup: So, what happens if we mix types in an operation?	33
5.7	Backup: And during assignment?	34
5.8	Backup: Primitive data types: Implicit conversion	35
5.9	Backup: Explicit conversion: Cast operator	36
5.10	Backup: With the Implicit or explicit conversion	36
5.11	Backup: And, when we cast different representation ?	37
5.12	Backup: Type conversion: Examples	38
5.13	Backup: Type conversion: Examples	39
5.14	Backup: Type conversion: Usual problems	40
5.15	Backup: Type conversion: Usual problems	41
6	Allocation, Qualifiers, Macros	42
6.1	Main Path	42
6.2	Worked Example	43
6.3	Anecdote Or Motivation	43
6.4	Check Question	44
6.5	Backup Index	44
6.6	Backup: Variable allocation and lifetime	47
6.7	Backup: Variable allocation and lifetime	48
6.8	Backup: Variable allocation and lifetime	49
6.9	Backup: Data objects: Qualifiers	50
6.10	Backup: Object-like Macros	50
6.11	Backup: Other reserved constants	51

1 Instructions and Comments

Live pacing: about 8 min

Question. What is the role of Instructions and Comments in the course story?

Core claim. Instructions and Comments is one step in the path from source text to reliable execution.

Target capability. Students can connect the topic to a concrete command, program state, or memory state.

1.1 Main Path

- Start from the question: What is the role of Instructions and Comments in the course story?
- Build the model: Instructions and Comments is one step in the path from source text to reliable execution.
- End with a checkable action: Students can connect the topic to a concrete command, program state, or memory state.
- Keep only this slide on the horizontal path during the live lecture.

- CM 4-5 Basic components of C language: Primitive data types, Libraries and Control structures

#148

1.2 Worked Example

- Use this as the live trace: Use one short trace, then ask students which state changed and which state did not.
- Representative source slide: 148
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

- CM 4-5 Basic components of C language: Primitive data types, Libraries and Control structures

#148

1.3 Anecdote Or Motivation

Keep the discussion anchored in observable behavior: command output, compiler message, or memory drawing.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

1.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Instructions & Comments

- Instruction is any expression finishing with ;
- Block of instructions
- {

#149

1.5 Backup Index

Original slides 148-149

- CM 4-5 Basic components of C language: Primitive data types, Libraries and Control structures

#148

slide 148

Instructions & Comments

- Instruction is any expression finishing with ;
- Block of instructions
- {

#149

slide 149

Anchor: CM 4-5 Basic components of C language: Primitive data types, Libraries and Control structures; Instructions & Comments

1.6 Backup: CM 4-5 Basic components of C language: Primitive data types, Libraries and Control structures

Original slide 148

- CM 4-5 Basic components of C language: Primitive data types, Libraries and Control structures

#148

1.7 Backup: Instructions & Comments

Original slide 149

- Instruction is any expression finishing with ;

- Block of instructions:
- {
- list of instructions
- }
- Comments
- /* this is a comment */
- // this is one line comment
- The language C does not support disambiguation
- The keywords of the language are reserved and cannot be used for naming variables and functions.

Instructions & Comments

- Instruction is any expression finishing with ;
- Block of instructions
- {

#149

2 Value Representation and Data Objects

Live pacing: about 8 min

Question. What is the role of Value Representation and Data Objects in the course story?

Core claim. Value Representation and Data Objects is one step in the path from source text to reliable execution.

Target capability. Students can connect the topic to a concrete command, program state, or memory state.

2.1 Main Path

- Start from the question: What is the role of Value Representation and Data Objects in the course story?
- Build the model: Value Representation and Data Objects is one step in the path from source text to reliable execution.
- End with a checkable action: Students can connect the topic to a concrete command, program state, or memory state.
- Keep only this slide on the horizontal path during the live lecture.

Value representation

- Hexadecimal numbers begin with 0x:
- 0x23 is 23₁₆ (which is 35 in the base of 10)
- Octal numbers begin with 0:

#150

2.2 Worked Example

- Use this as the live trace: Use one short trace, then ask students which state changed and which state did not.
- Representative source slide: 151
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Data object definition

- Syntax:
- type identifier;
- Function:
- Reserve in the memory a part equal to the size required to store a data object of the type type
- Associate the starting address of this memory part with the identifier

#151

2.3 Anecdote Or Motivation

Keep the discussion anchored in observable behavior: command output, compiler message, or memory drawing.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

2.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Defining primitive data objects: Examples

```
Definitions
int i;
float f1, f2;
unsigned char c1, c2;
Definitions with initialization
int i = 3;
float f1 = 1.2, f2 = 0.3;
unsigned char c1 = 'A', c2 = 65;
```

- What is value of these identifiers?
- Whatever is already stored in the memory just reserved...
- (except global and static variables)

- INITIALIZE ALL VARIABLES !!

- !!!

- Value of ASCII character

#156

2.5 Backup Index

Original slides 150-156

Value representation

- Hexadecimal numbers begin with 0x:
- 0x23 is 2316 (which is 35 in the base of 10)
- Octal numbers begin with 0:

#150

slide 150

Data object definition

- Syntax:
- type identifier;
- Function:
- Reserve in the memory a part equal to the size required to store a data object of the type type
- Associate the starting address of this memory part with the identifier

#151

slide 151

Primitive data types: Type specifiers

- Same as in JAVA:
- Integral
- char, short, int, long
- Real and complex
- float, double
- With additional specifiers for the representation:
- unsigned: always 0 or positive
- Representation: Binary
- signed: positive or negative:
- Representation: implementation depended
- 1's complement
- 2's complement (usual one)
- sign and magnitude
- Example: unsigned short int

#152

slide 152

Primitive data types: Size

The exact data type size is NOT part of the C standard
int can be 16 or 32 bits..
The standard only says that the range of a larger type shall be at least as big as the smaller type.
char short long long long
char: The minimum supported data type
Compiler decides based on the target platform architecture! (though, there is a minimum allowed width)
sizeof() can give the actual size in bytes
To give a specific storage size use the YintX_t type
Y = u for unsigned
X = 8, 16, 32 etc for the number of bits to be used

#153

slide 153

Primitive data types: Min values

Type	Storage size	Value range
char	usually signed (but not standard)	
unsigned char	1 byte	0 to 255
signed char	1 byte	(-127 to 127)
int	2 bytes (215)	(-2147483648 to 2147483647)
(usually 4 bytes)		
unsigned int	2 bytes	0 to 2147483647
(usually 4 bytes)		
short	2 bytes (215)	(-32768 to 32767)
unsigned short	2 bytes	0 to 2147483647
long	4 bytes (231)	(-2147483648 to 2147483647)
unsigned long	4 bytes	0 to 2147483647
long long	8 bytes (263)	(-263 to 263)
unsigned long long	8 bytes	0 to 264

#154

slide 154

Primitive data types: Float

Type	Storage size	Value range	Precision
float	4 byte	1.2E-38 to 3.4E+38	single
double	8 byte	2.3E-308 to 1.7E+308	double
long double	10 byte	3.4E-4932 to 1.1E+4932	extended

#155

slide 155

Anchors: Value representation; Data object definition; Primitive data types: Type specifiers; Primitive data types: Size; Primitive data types: Min values; Primitive data types: Float

2.6 Backup: Value representation

Original slide 150

- Hexadecimal numbers begin with 0x:
- 0x23 is 2316 (which is 35 in the base of 10)
- Octal numbers begin with 0:
- 023 is 238 (which is 19 in the base of 10)
- No prefix for decimal numbers:
- 23 is 2310
- Negative representation - operator
- -23 is the negation of 23

Value representation

- Hexadecimal numbers begin with 0x:
- 0x23 is 23₁₆ (which is 35 in the base of 10)
- Octal numbers begin with 0:

#150

2.7 Backup: Data object definition

Original slide 151

- Syntax:
- type identifier;
- Function:
- Reserve in the memory a part equal to the size required to store a data object of the type type
- Associate the starting address of this memory part with the identifier

Data object definition

- Syntax:
- type identifier;
- Function:
- Reserve in the memory a part equal to the size required to store a data object of the type type
- Associate the starting address of this memory part with the identifier

#151

2.8 Backup: Primitive data types: Type specifiers

Original slide 152

- Same as in JAVA:
- Integral
- char, short, int, long
- Real and complex
- float, double
- With additional specifiers for the representation:
- unsigned: always 0 or positive
- Representation: Binary
- signed: positive or negative:
- Representation: implementation depended
- 1's complement
- 2's complement (usual one)
- sign and magnitude
- Example: unsigned short int

Primitive data types: Type specifiers

- Same as in JAVA:
- Integral
- char, short, int, long
- Real and complex
- float, double
- With additional specifiers for the representation:
- unsigned: always 0 or positive
- Representation: Binary
- signed: positive or negative:
- Representation: implementation depended
- 1's complement
- 2's complement (usual one)
- sign and magnitude
- Example: unsigned short int

#152

2.9 Backup: Primitive data types: Size

Original slide 153

The exact data type size is NOT part of the C standard

int can be 16 or 32 bits...

The standard only says that the range of a larger type shall be at least as big as the smaller type.

char short long long long

char: The minimum supported data type

Compiler decides based on the target platform architecture! (though, there is a minimum allowed width)

sizeof() can give the actual size in bytes

To give a specific storage size use the YintX_t type

Y = u for unsigned

X = 8, 16, 32 etc for the number of bits to be used

Primitive data types: Size

The exact data type size is NOT part of the C standard
int can be 16 or 32 bits..
The standard only says that the range of a larger type shall be at least as big as the smaller type.
char short long long long
char: The minimum supported data type
Compiler decides based on the target platform architecture! (though, there is a minimum allowed width)
sizeof() can give the actual size in bytes
To give a specific storage size use the YintX_t type
Y = u for unsigned
X = 8, 16, 32 etc for the number of bits to be used

#153

2.10 Backup: Primitive data types: Min values

Original slide 154

Type	Storage	size	Value range
char	usually	signed	(but not standard)
unsigned char	1	byte	0 to 255
signed char	1	byte	(-128) to 127
int	2	bytes	(-2147483648) to 2147483647
	(usually 4	bytes)	
unsigned int	2	bytes	0 to 4294967295
	(usually 4	bytes)	
short	2	bytes	(-32768) to 32767
unsigned short	2	bytes	0 to 65535
long	4	bytes	(-2147483648) to 2147483647
unsigned long	4	bytes	0 to 4294967295
long long	8	bytes	(-9223372036854775808) to 9223372036854775807
unsigned long long	8	bytes	0 to 18446744073709551615

Primitive data types: Min values

Type	Storage size	Value range
char	usually signed (but not standard)	
unsigned char	1 byte	0 to 255
signed char	1 byte	(-128) to 127
int	2 bytes (215)	to 215
(usually 4 bytes)		
unsigned int	2 bytes	0 to 216
(usually 4 bytes)		
short	2 bytes (215)	to 215
unsigned short	2 bytes	0 to 216
long	4 bytes (231)	to 231
unsigned long	4 bytes	0 to 216
long long	8 bytes (263)	to 263
unsigned long long	8 bytes	0 to 264

#154

2.11 Backup: Primitive data types: Float

Original slide 155

- Type Storage size Value range Precision
- float 4 byte 1.2E-38 to 3.4E+38 single
- double 8 byte 2.3E-308 to 1.7E+308 double
- long double 10 byte 3.4E-4932 to 1.1E+4932 extended

Primitive data types: Float

Type	Storage size	Value range	Precision
float	4 byte	1.2E-38 to 3.4E+38	single
double	8 byte	2.3E-308 to 1.7E+308	double
long double	10 byte	3.4E-4932 to 1.1E+4932	extended

#155

2.12 Backup: Defining primitive data objects: Examples

Original slide 156

Definitions

```
int i;  
float f1, f2;  
unsigned char c1, c2;  
Definitions with initialization  
int i = 3;  
float f1 = 1.2, f2 = 0.3;  
unsigned char c1 = 'A', c2 = 65;
```

- ▶ What is value of these identifiers?
- ▶ Whatever is already stored in the memory just reserved...
- (except global and static variables)
- INITIALIZE ALL VARIABLES !!
- !!!
- Value of ASCII character

Defining primitive data objects: Examples

Definitions

```
int i;  
float f1, f2;  
unsigned char c1, c2;  
Definitions with initialization  
int i = 3;  
float f1 = 1.2, f2 = 0.3;  
unsigned char c1 = 'A', c2 = 65;
```

-- What is value of these identifiers?
-- Whatever is already stored in the memory just reserved..
(except global and static variables)

- INITIALIZE ALL VARIABLES !!

- !!!

- Value of ASCII character

#156

3 Operators

Live pacing: about 8 min

Question. What exactly is stored when a C variable has a type?

Core claim. A C type is a contract about size, interpretation, operations, conversion, and lifetime.

Target capability. Students can predict simple operator results and recognize dangerous implicit conversions.

3.1 Main Path

- Start from the question: What exactly is stored when a C variable has a type?
- Build the model: A C type is a contract about size, interpretation, operations, conversion, and lifetime.
- End with a checkable action: Students can predict simple operator results and recognize dangerous implicit conversions.
- Keep only this slide on the horizontal path during the live lecture.

Operators

Perform operations on data objects
They have priorities in the order they are applied:
Check the priority order
USE PARENTHESIS
Operator types
Arithmetic
Relational
Logical
Bitwise
Assignment
.....

#157

3.2 Worked Example

- Use this as the live trace: Use `int`, `unsigned`, and `float` values that print differently under `%d`, `%u`, and `%f`.
- Representative source slide: 158
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Arithmetic operators

- A = 10, B = 20

Operator Description Example
+ Adds two operands. A + B = 30
- Subtracts second operand from the first. A - B = -10
* Multiplies both operands. A * B = 200
/ Divides numerator by denominator. B / A = 2
% Remainder after integer division. B % A = 0
(assignment at the same time).
++ Increases the integer value by one. A++ = 11
-- Decreases the integer value by one. A-- = 9

- !!!

Postfix (Post-increment): Do the operation and return old value
A++ returns 10 and then does A=11
Prefix (Pre-increment): Do the operation and return new value
++A does =11 and then returns A

#158

3.3 Anecdote Or Motivation

The machine stores bits; the type tells the compiler and programmer how to read those bits.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

3.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Let's...

- Kahoot time !
- (6 Q)

#164

3.5 Backup Index

Original slides 157-164

Operators

Perform operations on data objects
They have priorities in the order they are applied:
Check the priority order
USE PARENTHESIS
Operator types
Arithmetic
Relational
Logical
Bitwise
Assignment
.....

#157

slide 157

Arithmetic operators

- A = 10, B = 20

Operator Description Example

+ Adds two operands. A + B = 30

- Subtracts second operand from the first. A - B = -10

* Multiplies both operands. A * B = 200

/ Divides numerator by denominator. B / A = 2

% Remainder after integer division. B % A = 0
(assignment at the same time).

++ Increases the integer value by one. A++ = 11

-- Decreases the integer value by one. A-- = 9

- !!!

Postfix (Post-increment): Do the operation and return old value

A++ returns 10 and then does A=11

Prefix (Pre-increment): Do the operation and return new value

++A does =11 and then returns A

#158

slide 158

Relational operators

- A = 10, B = 20

Op. Description

== Checks if the values of two operands are equal.
If yes, the condition becomes true.

!= Checks if the values of two operands are not equal
different. If yes, the condition becomes true.

> Checks if the value of left operand is greater than the
value of right operand. If yes, it becomes true.

< Checks if the value of left operand is less than the
value of right operand. If yes, it becomes true.

>= Checks if the value of left operand is greater than or
equal to the value of right operand.

<= Checks if the value of left operand is less than or
equal to the value of right operand.

- Example
- (A == B) is not true.
- (A != B) is true.
- (A > B) is not true.
- (A < B) is true.
- (A >= B) is not true.
- (A <= B) is true

- Any integer value can be considered as Boolean
- A FALSE expression is coded as a zero integer
- A TRUE expression is coded as any non-zero integer value

#159

slide 159

Logical operators

- The operands are TRUE or FALSE

- A = 1, B = 0

Op.
&&
||
!

- Description
- Logical AND: If both the operands are non-zero, then the condition becomes true.
- Does not evaluate 2nd operand, if the first is false
- Logical OR: If any of the two operands is non-zero, then the condition becomes true.
- Does not evaluate 2nd operand, if the first is true
- Logical NOT: It is used to reverse the logical state of its operand. If a condition is true, then Logical NOT operator will make it false.

- Example
- (A && B) is false.
- (A || B) is true.
- !(A && B) is true.

#160

slide 160

Bitwise operators

- The operands are integral (short, int, unsigned, etc)

- A = 0x3 (0011) B = 0x2 (0010)

- Op. Description Example

- &
- |
- ^
- ~
- <<
- >>
- Binary AND: Copies a bit to the result, if it exists in both operands.
- Binary OR: Copies a bit if it exists in either operand.
- Binary XOR: Copies the bit if it is set only in one operand but not both.
- Binary Ones Complement: Has the effect of 'flipping' bits.
- Binary Left Shift: The left operand's value is moved left by the number of bits specified by the right operand.
- Binary Right Shift: The left operand's value is moved right by the number of bits specified by the right operand.

- (A & B) = 0x2
- (0010)

- (A | B) = 0x3
- (0011)

- (A ^ B) = 0x1
- (0001)

- (~A) = 0xC
- (1100)

- (A < 2) = 0xC
- (1100)

- (A > 2) = 0x0
- (0000)

#161

slide 161

Assignment operators		
		- A = 10, B = 20
Op.	Description	Example
=	Assignment operator. Assigns values from right side to left side operand	- C = A + B - C = 30
+=	combined with the other arithmetic operators (+, -, *, /, %).	
-=	combined with the other arithmetic operators (+, -, *, /, %).	
*=	combined with the other arithmetic operators (+, -, *, /, %).	
/=	combined with the other arithmetic operators (+, -, *, /, %).	
%=	combined with the other arithmetic operators (+, -, *, /, %).	
&=	combined with the other arithmetic operators (+, -, *, /, %).	- A += 1
=	combined with the other arithmetic operators (+, -, *, /, %).	- A = 11
^=	combined with the other arithmetic operators (+, -, *, /, %).	
<<=	combined with the other arithmetic operators (+, -, *, /, %).	
>>=	combined with the other arithmetic operators (+, -, *, /, %).	

#162

slide 162

Anchors: Operators; Arithmetic operators; Relational operators; Logical operators; Bitwise operators; Assignment operators

3.6 Backup: Operators

Original slide 157

```

Perform operations on data objects
They have priorities in the order they are applied:
Check the priority order
USE PARENTHESIS
Operator types
Arithmetic
Relational
Logical
Bitwise
Assignment
.....

```

Operators

Perform operations on data objects
They have priorities in the order they are applied:
Check the priority order
USE PARENTHESES
Operator types
Arithmetic
Relational
Logical
Bitwise
Assignment
.....

#157

3.7 Backup: Arithmetic operators

Original slide 158

- A = 10, B = 20

Operator Description Example

+ Adds two operands. A + B = 30

Subtracts second operand from the first. A - B = -10

* Multiplies both operands. A * B = 200

/ Divides numerator by denominator. B / A = 2

% Remainder after integer division. B % A = 0

(assignment at the same time).

++ Increases the integer value by one A++ = 11

-- Decreases the integer value by one. A-- = 9

- !!!

Postfix (Post-increment): Do the operation and **return** old value

A++ returns 10 and then does A=11

Prefix (Pre-increment): Do the operation and **return** new value

++A does =11 and then returns A

Arithmetic operators

- A = 10, B = 20

Operator Description Example
+ Adds two operands. A + B = 30
- Subtracts second operand from the first. A - B = -10
* Multiplies both operands. A * B = 200
/ Divides numerator by denominator. B / A = 2
% Remainder after integer division. B % A = 0
(assignment at the same time).
++ Increases the integer value by one. A++ = 11
-- Decreases the integer value by one. A-- = 9

- !!!

Postfix (Post-increment): Do the operation and return old value
A++ returns 10 and then does A=11
Prefix (Pre-increment): Do the operation and return new value
++A does =11 and then returns A

#158

3.8 Backup: Relational operators

Original slide 159

- A = 10, B = 20

Op. Description

== Checks **if** the values of two operands are equal.

If yes, the condition becomes **true**.

!= Checks **if** the values of two operands are not equal
different. If yes, the condition becomes **true**.

> Checks **if** the value of left operand is greater than the
value of right operand. If yes, it becomes **true**.

< Checks **if** the value of left operand is less than the
value of right operand. If yes, it becomes **true**.

>= Checks **if** the value of left operand is greater than or
equal to the value of right operand.

<= Checks **if** the value of left operand is less than or
equal to the value of right operand.

- Example
- (A == B) is not true.
- (A != B) is true.
- (A > B) is not true.
- (A < B) is true.
- (A >= B) is not true.
- (A <= B) is true
- Any integer value can be considered as Boolean
- A FALSE expression is coded as a zero integer
- A TRUE expression is coded as any non-zero integer value

Relational operators

Op. Description

`==` Checks if the values of two operands are equal. If yes, the condition becomes true.

`!=` Checks if the values of two operands are not equal different. If yes, the condition becomes true.

`>` Checks if the value of left operand is greater than the value of right operand. If yes, it becomes true.

`<` Checks if the value of left operand is less than the value of right operand. If yes, it becomes true.

`>=` Checks if the value of left operand is greater than or equal to the value of right operand.

`<=` Checks if the value of left operand is less than or equal to the value of right operand.

- A = 10, B = 20

- Example

- `(A == B)` is not true.

- `(A != B)` is true.

- `(A > B)` is not true.

- `(A < B)` is true.

- `(A >= B)` is not true.

- `(A <= B)` is true

- Any integer value can be considered as Boolean

- A FALSE expression is coded as a zero integer

- A TRUE expression is coded as any non-zero integer value

#159

3.9 Backup: Logical operators

Original slide 160

- The operands are TRUE or FALSE
- A = 1, B = 0
- Example
- `(A && B)` is false.
- `(A || B)` is true.
- `!(A && B)` is true.

Op.

`&&`

`||`

`!`

- Description
- Logical AND: If both the operands are non-zero, then the condition becomes true.
- Does not evaluate 2nd operand, if the first is false
- Logical OR: If any of the two operands is non-zero, then the condition becomes true.
- Does not evaluate 2nd operand, if the first is true
- Logical NOT: It is used to reverse the logical state of its operand. If a condition is true, then Logical NOT operator will make it false.

Logical operators

- The operands are TRUE or FALSE

- A = 1, B = 0

Op.
&
||
!

- Description
- Logical AND: If both the operands are non-zero, then the condition becomes true.
- Does not evaluate 2nd operand, if the first is false
- Logical OR: If any of the two operands is non-zero, then the condition becomes true.
- Does not evaluate 2nd operand, if the first is true
- Logical NOT: It is used to reverse the logical state of its operand. If a condition is true, then Logical NOT operator will make it false.

- Example
- (A & B) is false.
- (A || B) is true.
- !(A & B) is true.

#160

3.10 Backup: Bitwise operators

Original slide 161

- The operands are integral (short, int, unsigned, etc)
- A = 0x3 (0011) B = 0x2 (0010)
- Op. Description Example
- Binary AND: Copies a bit to the result, if it exists in both operands.
- Binary OR: Copies a bit if it exists in either operand.
- Binary XOR: Copies the bit if it is set only in one operand but not both.
- Binary Ones Complement: Has the effect of 'flipping' bits.
- Binary Left Shift: The left operands value is moved left by the number of bits specified by the right operand.
- Binary Right Shift: The left operands value is moved right by the number of bits specified by the right operand.
- (A & B) = 0x2
- (10)
- &
- |
- ^
- ~
- <<
- >>
- (A | B) = 0x3
- (11)
- (A ^ B) = 0x1

- (1)
- ($\sim A$) = 0xC
- (1100)
- ($A < < 2$) = 0xC
- (1100)
- ($A > > 2$) = 0x0
- (0)

Bitwise operators		
- The operands are integral (short, int, unsigned, etc)		
- A = 0x3 (0011) B = 0x2 (0010)		
Op.	Description	Example
&	- Binary AND: Copies a bit to the result, if it exists in both operands.	- ($A \& B$) = 0x2 - (0010)
	- Binary OR: Copies a bit if it exists in either operand.	- ($A B$) = 0x3 - (0011)
^	- Binary XOR: Copies the bit if it is set only in one operand but not both.	- ($A \wedge B$) = 0x1 - (0001)
~	- Binary Ones Complement: Has the effect of 'flipping' bits.	- ($\sim A$) = 0xC - (1100)
<<	- Binary Left Shift: The left operand's value is moved left by the number of bits specified by the right operand.	- ($A < < 2$) = 0xC - (1100)
>>	- Binary Right Shift: The left operand's value is moved right by the number of bits specified by the right operand.	- ($A > > 2$) = 0x0 - (0000)

#161

3.11 Backup: Assignment operators

Original slide 162

- A = 10, B = 20
- Op. Description Example
- Simple assignment operator. Assigns values from right side operands to left side operand
- In can be combined with the other arithmetic operators (+, -, *, /, %) and bitwise operators (&, |, ^, <<, >>).
- It performs the arithmetic operation with right operand and the left operand and assign the result to the left operand.
- C = A + B
- C = 30

=
+=
-=
*=
/=

%=

&=

|=
^=
<<=
>>=

- A += 1
- A = 11

Assignment operators		
Op.	Description	Example
=	Assignment operator. Assigns values from right side to left side operand	- A = 10, B = 20
+=	combined with the other arithmetic operators (+, -, *, /, %).	- C = A + B
-=	combined with the other arithmetic operators (+, -, *, /, %).	- C = 30
*=	combined with the other arithmetic operators (+, -, *, /, %).	
/=	combined with the other arithmetic operators (+, -, *, /, %).	
%=	combined with the other arithmetic operators (+, -, *, /, %).	
&=	combined with the other arithmetic operators (+, -, *, /, %).	- A += 1
=	combined with the other arithmetic operators (+, -, *, /, %).	- A = 11
^=	combined with the other arithmetic operators (+, -, *, /, %).	
<<=	combined with the other arithmetic operators (+, -, *, /, %).	
>>=	combined with the other arithmetic operators (+, -, *, /, %).	

#162

3.12 Backup: Common error: Assignment Vs Testing

Original slide 163

- Assignment: `nom_variable = expression ;`
- `x=3;`
- It is an expression that stores the value 3 to the variable x
- Test of equality: `expression1 == expression2 ;`
- `x==3;`
- It returns 1 if the stored value in x is equal to 3, otherwise 0
- ATTENTION:
- **4 and == are different !**

Common error: Assignment Vs Testing

- Assignment: `nom_variable = expression;`
- `x=3;`
- It is an expression that stores the value 3 to the variable x

- ATTENTION:
- `=` and `==` are different !

#163

4.1 Backup: Let's...

Original slide 164

- Kahoot time !
- (6 Q)

Let's...

- Kahoot time !
- (6 Q)

#164

5 Conversions

Live pacing: about 8 min

Question. What exactly is stored when a C variable has a type?

Core claim. A C type is a contract about size, interpretation, operations, conversion, and lifetime.

Target capability. Students can predict simple operator results and recognize dangerous implicit conversions.

5.1 Main Path

- Start from the question: What exactly is stored when a C variable has a type?
- Build the model: A C type is a contract about size, interpretation, operations, conversion, and lifetime.
- End with a checkable action: Students can predict simple operator results and recognize dangerous implicit conversions.
- Keep only this slide on the horizontal path during the live lecture.

So, what happens if we mix types in an operation?

- a (operator) b type a = type b

- Automatic IMPLICIT and SILENT conversion!
- Both operands (a, b) must be data objects of the same type
- Converted to the common higher rank of the operands based on the type width:
- PROMOTION
- Integer promotions
- signed -> unsigned
- small integer types: char, short, int16, int8 etc (BOTH signed and unsigned) are converted to
- int: IF the range of int can represent ALL values of the original type
- else: unsigned int

#165

5.2 Worked Example

- Use this as the live trace: Use `int`, `unsigned`, and `float` values that print differently under `%d`, `%u`, and `%f`.
- Representative source slide: 174
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

```
Casting a larger type to a smaller one (DEMOTION):  
Truncation: Cuts off the MSB (overflow occurs).  
int f = 0x12345678;  
short int g;  
g = f;  
Comparisons, especially with unsigned, char and short!  
unsigned int a = 0;  
if (a > -1)  
    printf("TRUE\n");  
else  
    printf("FALSE\n");
```

- SECURITY ISSUES

#174

5.3 Anecdote Or Motivation

The machine stores bits; the type tells the compiler and programmer how to read those bits.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

5.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

```
Casting a larger type to a smaller one (DEMOTION):  
Truncation: Cuts off the MSB (overflow occurs).  
int f = 0x12345678;  
short int g;  
g = f;  
Comparisons, especially with unsigned, char and short!  
unsigned int a = 0;  
if (a > -1)  
    printf("TRUE\n");  
else  
    printf("FALSE\n");
```

- SECURITY ISSUES

#174

5.5 Backup Index

Original slides 165-174

So, what happens if we mix types in an operation?

- a (operator) b type a = type b

- Automatic IMPLICIT and SILENT conversion!
- Both operands (a, b) must be data objects of the same type
- Converted to the common higher rank of the operands based on the type width:
- PROMOTION
- Integer promotions:
- signed -> unsigned
- small integer types: char, short, int16, int8 etc (BOTH signed and unsigned) are converted to
- int: IF the range of int can represent ALL values of the original type
- else: unsigned int

#165

slide 165

And during assignment?

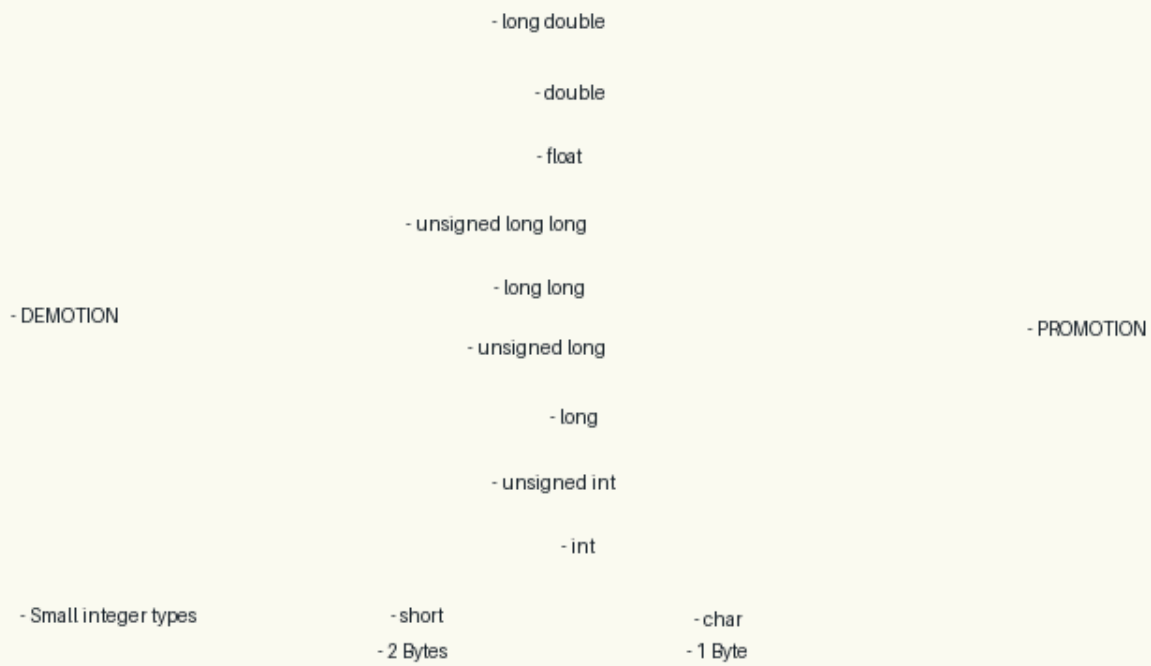
c = a (operator) b
type c = type (result of operation)

- Still automatic IMPLICIT and SILENT conversion!
- Converted to the type of data object that stores the result
- Higher rank: PROMOTION
- Lower rank: DEMOTION

#166

slide 166

Primitive data types: Implicit conversion



#167

slide 167

Explicit conversion: Cast operator

- Syntax
- (type) identifier;
- Creates a copy of the data object and changes its type.
- The data object of the identifier is NOT modified

#168

slide 168

With the Implicit or explicit conversion

- The type changes ->
- So, the storage size and/or the representation changes ->
- So, the value CAN change

#169

slide 169

And, when we cast different representation ?

- Casting float to int: It changes representation!
- float is represented in IEEE 754 single precision
- int is usually 2's Complement
- float f = 3.9;
- int val = (int) f;
- Value gets truncated !
- 3.9 is truncated to 3.0 and stored as 3 in 2's complement
- Casting int to float: It changes representation !
- int f = 3;
- float val = (float) f;
- Value to convert to float (thus, 3 is converted to 3.0)

#170

slide 170

Anchors: So, what happens if we mix types in an operation?; And during assignment?; Primitive data types: Implicit conversion; Explicit conversion: Cast operator; With the Implicit or explicit conversion; And, when we cast different representation ?

5.6 Backup: So, what happens if we mix types in an operation?

Original slide 165

- a (operator) b type a = type b
- Automatic IMPLICIT and SILENT conversion!

- Both operands (a, b) must be data objects of the same type
- Converted to the common higher rank of the operands based on the type width:
- PROMOTION
- Integer promotions:
- signed -> unsigned
- small integer types: char, short, int16, int8 etc (BOTH signed and unsigned) are converted to
- int : IF the range of int can represent ALL values of the original type
- else: unsigned int

So, what happens if we mix types in an operation?

- a (operator) b type a = type b

- Automatic IMPLICIT and SILENT conversion!
 - Both operands (a, b) must be data objects of the same type
 - Converted to the common higher rank of the operands based on the type width:
 - PROMOTION
 - Integer promotions:
 - signed -> unsigned
 - small integer types: char, short, int16, int8 etc (BOTH signed and unsigned) are converted to
 - int: IF the range of int can represent ALL values of the original type
 - else: unsigned int

#165

5.7 Backup: And during assignment?

Original slide 166

- c = a (operator) b
- type c = type (result of operation)
-Still automatic IMPLICIT and SILENT conversion!
- Converted to the type of data object that stores the result:
- Higher rank: PROMOTION
- Lower rank: DEMOTION

And during assignment?

c = a (operator) b
type c = type (result of operation)

- Still automatic IMPLICIT and SILENT conversion!
- Converted to the type of data object that stores the result
- Higher rank: PROMOTION
- Lower rank: DEMOTION

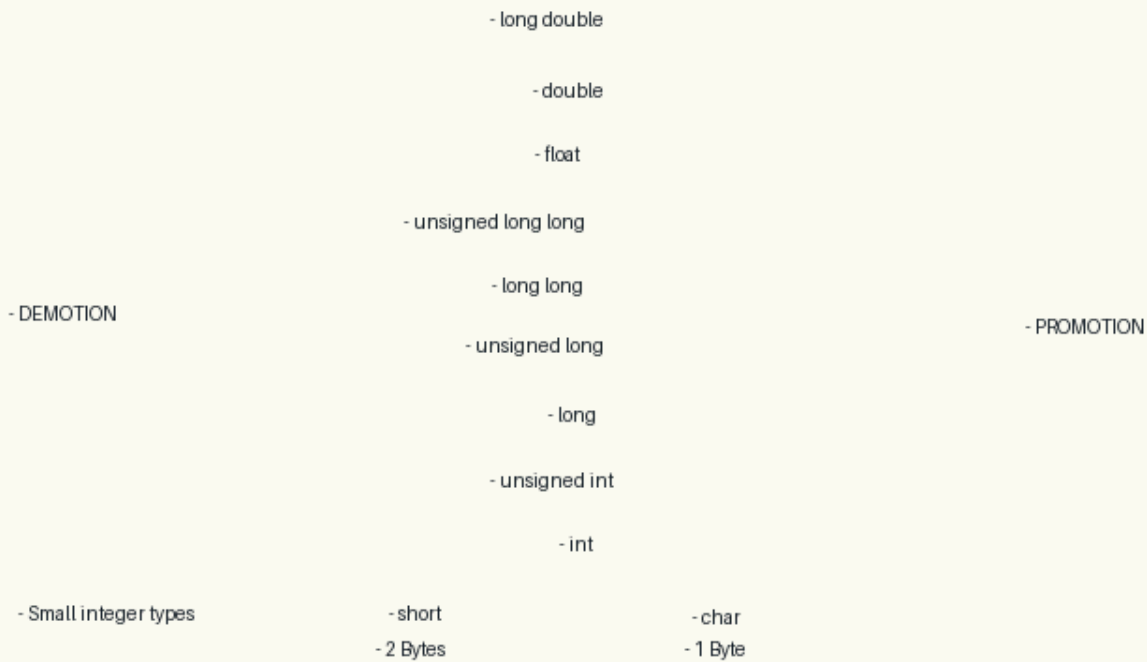
#166

5.8 Backup: Primitive data types: Implicit conversion

Original slide 167

- long double
- double
- float
- unsigned long long
- long long
- DEMOTION
- PROMOTION
- unsigned long
- long
- unsigned int
- int
- Small integer types
- short
- char
- 2 Bytes
- 1 Byte

Primitive data types: Implicit conversion



#167

5.9 Backup: Explicit conversion: Cast operator

Original slide 168

- Syntax
- (type) identifier;
- Creates a copy of the data object and changes its type.
- The data object of the identifier is NOT modified

Explicit conversion: Cast operator

- Syntax
- (type) identifier;
- Creates a copy of the data object and changes its type.
- The data object of the identifier is NOT modified

#168

5.10 Backup: With the Implicit or explicit conversion

Original slide 169

- The type changes ->

- So, the storage size and/or the representation changes ->
- So, the value CAN change

With the Implicit or explicit conversion

- The type changes ->
- So, the storage size and/or the representation changes ->
- So, the value CAN change

#169

5.11 Backup: And, when we cast different representation ?

Original slide 170

- Casting float to int: It changes representation!
- float is represented in IEEE 754 single precision
- int is usually 2's Complement
- float f = 3.9 ;
- int val = (int) f ;
- Value gets truncated !
- 3.9 is truncated to 3.0 and stored as 3 in 2's complement
- Casting int to float: It changes representation !
- int f = 3 ;
- float val = (float) f ;
- Value to convert to float (thus, 3 is converted to 3.0)

And, when we cast different representation ?

- Casting float to int: It changes representation!
- float is represented in IEEE 754 single precision
- int is usually 2's Complement
- float f = 3.9;
- int val = (int) f;
- Value gets truncated !
- 3.9 is truncated to 3.0 and stored as 3 in 2's complement
- Casting int to float: It changes representation !
- int f = 3;
- float val = (float) f;
- Value to convert to float (thus, 3 is converted to 3.0)

#170

5.12 Backup: Type conversion: Examples

Original slide 171

- int x;
- Implicit conversion: $x = 1.5 + 1.6$;
- 1.5 -> double
- 1.6 -> double
- $1.5 + 1.6 = 3.1$ double addition
- x -> int
- 3.1 -> 3 DEMOTION to match with int
- $x = 3$
- Explicit conversion: $x = (\text{int}) 1.5 + (\text{int}) 1.6$;
- 1.5 -> double
- $(\text{int}) 1.5 \rightarrow 1$ DEMOTION
- 1.6 -> double
- $(\text{int}) 1.6 \rightarrow 1$ DEMOTION
- $1 + 1 = 2$ integer addition
- x -> int
- $x = 2$
- !!!

Type conversion: Examples

```
- int x;
- Implicit conversion: x = 1.5 + 1.6;
- 1.5 -> double
- 1.6 -> double
- 1.5 + 1.6 = 3.1 double addition
- x -> int - !!!
- 3.1 -> 3 DEMOTION to match with int
- x = 3
- Explicit conversion: x = (int) 1.5 + (int) 1.6;
- 1.5 -> double
- (int) 1.5 -> 1 DEMOTION
- 1.6 -> double
- (int) 1.6 -> 1 DEMOTION
- 1 + 1 = 2 integer addition
- x -> int
- x = 2
```

#171

5.13 Backup: Type conversion: Examples

Original slide 172

- float x;
- Implicit conversion: x = 3/5;
- 3 -> int
- 5 -> int
- 3/5 = 0 integer division
- x -> float
- 0.0 PROMOTION to match with float
- x = 0.0
- Explicit conversion x = (float) 3/5;
- 3 -> int
- (float) 3 -> 3.0 PROMOTION
- Promotion 5 -> 5.0 PROMOTION
- 3.0/5.0 = 0 float division
- x -> float
- x = 0.6

Type conversion: Examples

- float x;
- Implicit conversion: $x = 3/5$;
- $3 \rightarrow \text{int}$
- $5 \rightarrow \text{int}$
- $3/5 = 0$ integer division
- $x \rightarrow \text{float}$
- 0.0 PROMOTION to match with float
- $x = 0.0$
- Explicit conversion $x = (\text{float}) 3/5$;
- $3 \rightarrow \text{int}$
- $(\text{float}) 3 \rightarrow 3.0$ PROMOTION
- Promotion $5 \rightarrow 5.0$ PROMOTION
- $3.0/5.0 = 0$ float division
- $x \rightarrow \text{float}$
- $x = 0.6$

#172

5.14 Backup: Type conversion: Usual problems

Original slide 173

Casting signed to unsigned:

Negative value is converted to a high positive value

Especially occurs during function calls or return values

Casting a smaller signed int type to a larger unsigned int type

Sign extension: The sign bit is used in the unused bits, i.e. the Most Significant Bit (MSB) of the larger type

char and short are signed!

Negative value is converted to a high positive value

- SECURITY ISSUES

Type conversion: Usual problems

Casting signed to unsigned:
Negative value is converted to a high positive value
Especially occurs during function calls or return values
Casting a smaller signed int type to a larger unsigned int type
Sign extension: The sign bit is used in the unused bits, i.e. the Most Significant Bit (MSB) of the larger type
char and short are signed!
Negative value is converted to a high positive value

- SECURITY ISSUES

#173

5.15 Backup: Type conversion: Usual problems

Original slide 174

```
Casting a larger type to a smaller one (DEMOTION):  
Truncation: Cuts off the MSB (overflow occurs).  
int f = 0x12345678 ;  
short int g;  
g = f;  
Comparisons, especially with unsigned, char and short !  
unsigned int a = 0 ;  
if (a > -1)  
    printf("TRUE\n");  
else  
    printf("FALSE\n");
```

- SECURITY ISSUES

```
Casting a larger type to a smaller one (DEMOTION):  
Truncation: Cuts off the MSB (overflow occurs).  
int f = 0x12345678;  
short int g;  
g = f;  
Comparisons, especially with unsigned, char and short!  
unsigned int a = 0;  
if (a > -1)  
    printf("TRUE\n");  
else  
    printf("FALSE\n");
```

- SECURITY ISSUES

#174

6 Allocation, Qualifiers, Macros

Live pacing: about 8 min

Question. What does the operating system do between the program and the machine?

Core claim. The OS gives a program controlled access to files, processes, memory, and devices.

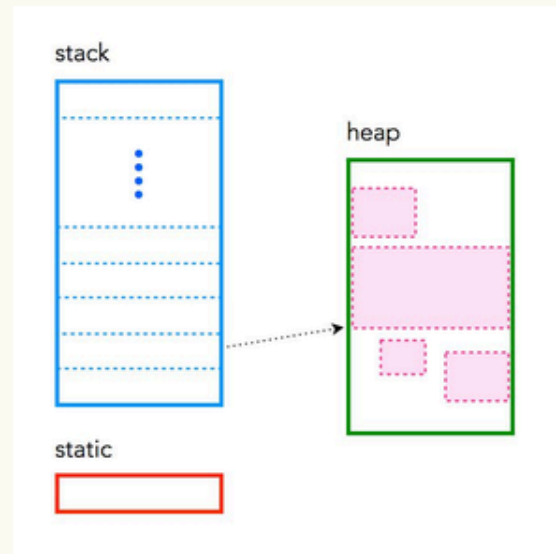
Target capability. Students can place shell commands, processes, files, and permissions in the OS layer.

6.1 Main Path

- Start from the question: What does the operating system do between the program and the machine?
- Build the model: The OS gives a program controlled access to files, processes, memory, and devices.
- End with a checkable action: Students can place shell commands, processes, files, and permissions in the OS layer.
- Keep only this slide on the horizontal path during the live lecture.

Variable allocation and lifetime

- Static memory:
- Variable Lifetime: complete program
- Stores
- Global variables: data objects declared outside of all function blocks
- Variables defined as static
- Global and static variables are initialized to zero by default by the compiler
- Allocation at compile/link time



#175

6.2 Worked Example

- Use this as the live trace: Compare `ls`, an editor, and a compiled program: all are processes asking the OS for services.
- Representative source slide: 179
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Object-like Macros

- Syntax: Through pre-processor directive `#define`
- `#define IDENTIFIER value`
- Function:
- Gives a symbolic name to a data object
- Convention: Symbolic names are given in UPPERCASE
- Examples
- `#define PI 3.1416`
- `#define NOM foo`

#179

6.3 Anecdote Or Motivation

A useful mental image: user programs are guests; the OS is the host that owns the keys.

- Use this only if students need a reason to care.

- If the room is already following, skip down to the check question.

6.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Other reserved constants

- Escape sequences
- The character '\n' is a new line (on Unix systems)
- The character '\b' is a backspace
- The character '\t' is a tab space
- The character '\0' is null character
- The double quotes ... defines a string of characters
- There is NO string type in C
- String = array of characters that ends with the null character ('\0')
- The simple quote ' defines a character

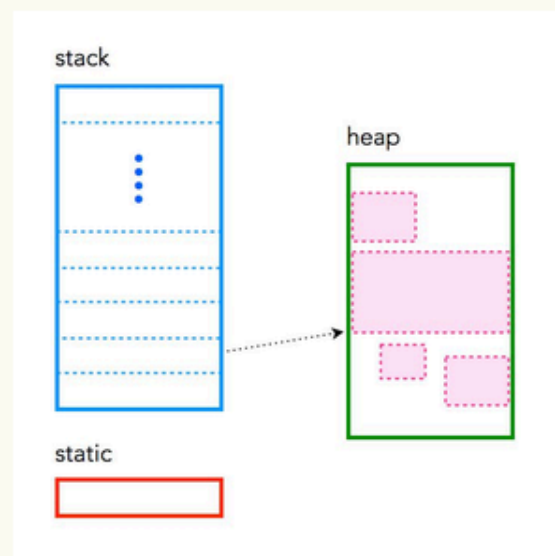
#180

6.5 Backup Index

Original slides 175-180

Variable allocation and lifetime

- Static memory:
- Variable Lifetime: complete program
- Stores
- Global variables: data objects declared outside of all function blocks
- Variables defined as static
- Global and static variables are initialized to zero by default by the compiler
- Allocation at compile/link time



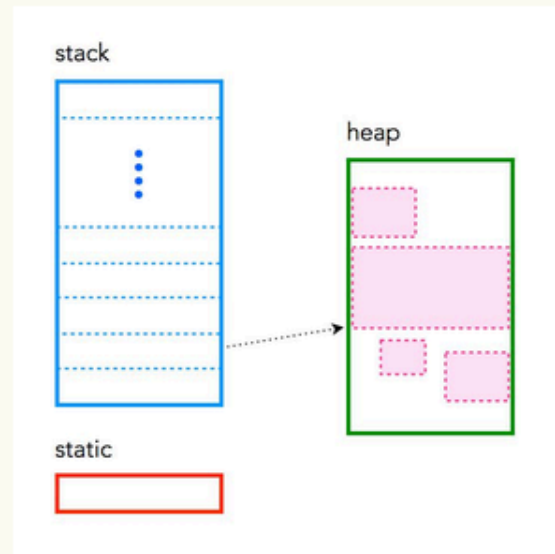
#175

slide 175

Variable allocation and lifetime

- !!!

- Stack:
- Variable Lifetime: specific block
- Stores
- Local variables: data objects declared within a block
- Variables defined as auto (by default the local ones)
- Variables defined as register, but the compiler couldn't store them to registers
- Allocated in an
- Automatic way
- Continuous way
- Allocation during execution



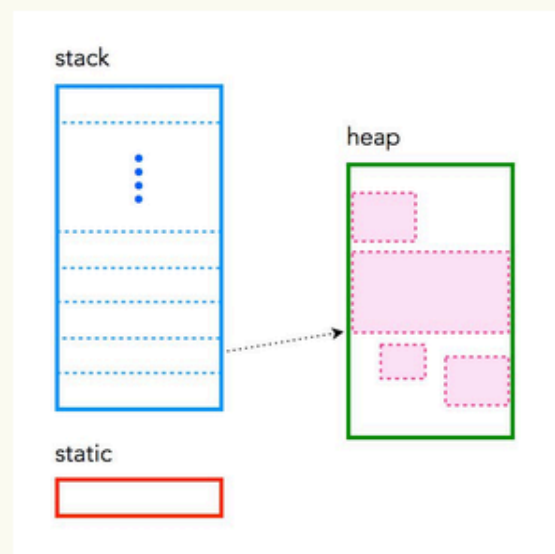
#176

slide 176

Variable allocation and lifetime

- !!!

- Heap:
- Variable lifetime: Managed by the program
- Large pool of memory
- Not allocated in contiguous order
- Dynamic allocation
- Allocation during execution



#177

slide 177

Data objects: Qualifiers

- Express additional information about a value
- Ensure the correct use of the data object
- `const`: The value of the data object does not change once it has been initialized

#178

slide 178

Object-like Macros

- Syntax: Through pre-processor directive `#define`
- `#define IDENTIFIER value`
- Function:
 - Gives a symbolic name to a data object
- Convention: Symbolic names are given in UPPERCASE
- Examples
 - `#define PI 3.1416`
 - `#define NOM foo`

#179

slide 179

Other reserved constants

- Escape sequences
- The character '\n' is a new line (on Unix systems)
- The character '\b' is a backspace
- The character '\t' is a tab space
- The character '\0' is null character
- The double quotes... defines a string of characters
- There is NO string type in C
- String = array of characters that ends with the null character ('\0')
- The simple quote ' defines a character

#180

slide 180

anchors: Variable allocation and lifetime; Data objects: Qualifiers; Object-like Macros; Other reserved constants

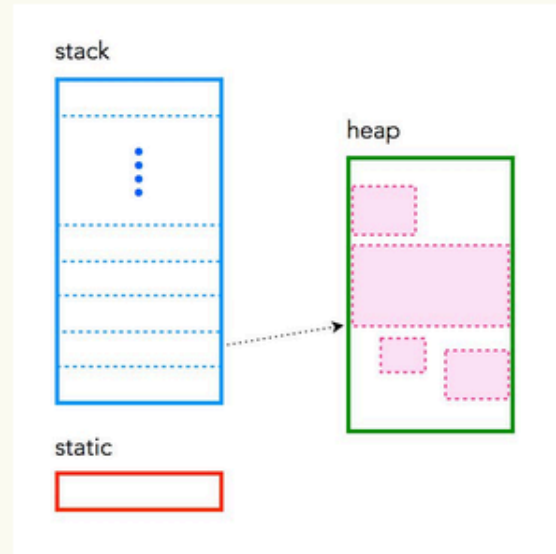
6.6 Backup: Variable allocation and lifetime

Original slide 175

- Static memory:
- Variable Lifetime: complete program
- Stores:
- Global variables: data objects declared outside of all function blocks
- Variables defined as static
- Global and static variables are initialized to zero by default by the compiler
- Allocation at compile/link time

Variable allocation and lifetime

- Static memory:
- Variable Lifetime: complete program
- Stores
- Global variables: data objects declared outside of all function blocks
- Variables defined as static
- Global and static variables are initialized to zero by default by the compiler
- Allocation at compile/link time



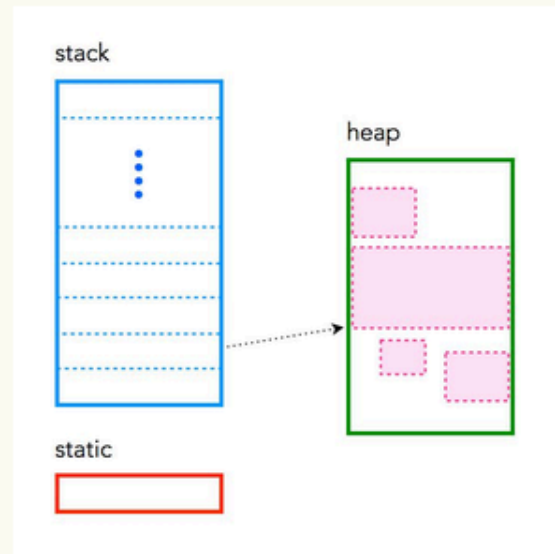
#175

6.7 Backup: Variable allocation and lifetime

Original slide 176

- !!!
- Stack:
- Variable Lifetime: specific block
- Stores
- Local variables: data objects declared within a block
- Variables defined as auto (by default the local ones)
- Variables defined as register, but the compiler couldn't store them to registers
- Allocated in an
- Automatic way
- Continuous way
- Allocation during execution

- Stack:
- Variable Lifetime: specific block
- Stores
- Local variables: data objects declared within a block
- Variables defined as auto (by default the local ones)
- Variables defined as register, but the compiler couldn't store them to registers
- Allocated in an
- Automatic way
- Continuous way
- Allocation during execution



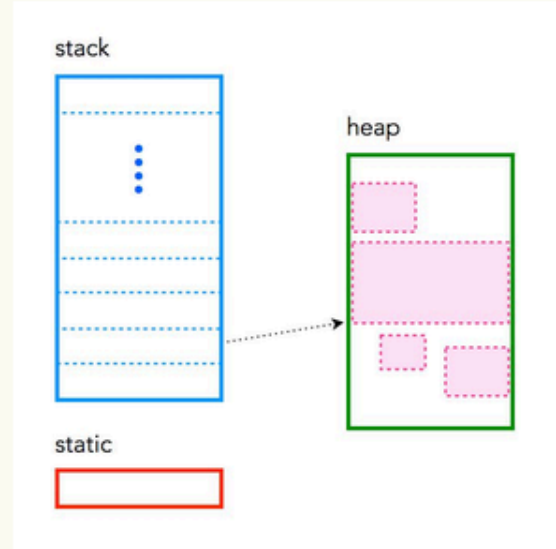
#176

6.8 Backup: Variable allocation and lifetime

Original slide 177

- !!!
- Heap:
- Variable lifetime: Managed by the program
- Large pool of memory
- Not allocated in contiguous order
- Dynamic allocation
- Allocation during execution

- Heap:
- Variable lifetime: Managed by the program
- Large pool of memory
- Not allocated in contiguous order
- Dynamic allocation
- Allocation during execution



#177

6.9 Backup: Data objects: Qualifiers

Original slide 178

- Express additional information about a value
- Ensure the correct use of the data object
- `const`: The value of the data object does not change once it has been initialized
- `volatile`: The compiler does not optimize, the data object is always accessed from the memory
- `restrict`: The only way to access a data object is through the pointer that points to it

Data objects: Qualifiers

- Express additional information about a value
- Ensure the correct use of the data object
- `const`: The value of the data object does not change once it has been initialized

#178

6.10 Backup: Object-like Macros

Original slide 179

- Syntax: Through pre-processor directive #define
- #define IDENTIFIER value
- Function:
- Gives a symbolic name to a data object
- Convention: Symbolic names are given in UPPERCASE
- Examples
- #define PI 3.1416
- #define NOM foo

Object-like Macros

- Syntax: Through pre-processor directive #define
- #define IDENTIFIER value
- Function:
- Gives a symbolic name to a data object
- Convention: Symbolic names are given in UPPERCASE
- Examples
- #define PI 3.1416
- #define NOM foo

#179

6.11 Backup: Other reserved constants

Original slide 180

- Escape sequences:
- The character '\n' is a new line (on Unix systems)
- The character '\b' is a backspace
- The character '\t' is a tab space
- The character '\0' is null character
- The double quotes ... defines a string of characters
- There is NO string type in C
- String = array of characters that ends with the null character (\0)
- The simple quote ' ' defines a character

Other reserved constants

- Escape sequences
- The character '\n' is a new line (on Unix systems)
- The character '\b' is a backspace
- The character '\t' is a tab space
- The character '\0' is null character
- The double quotes... defines a string of characters
- There is NO string type in C
- String = array of characters that ends with the null character ('\0')
- The simple quote '...' defines a character

#180